

Memorando

De: Emanuel Carneiro dos Santos

Nº de Matrícula: 20230429

Disciplina: Computação Paralela e Distribuída

Assunto: Introdução ao Ambiente Unix

Data: 23/03/2026

1. Introdução

Este laboratório teve como objetivo introduzir o desenvolvimento de aplicações em linguagem C no ambiente Unix, com foco na utilização de ferramentas fundamentais como o compilador gcc, o debugger gdb e o sistema de automação make.

Adicionalmente, exploraram-se conceitos fundamentais de sistemas operativos, nomeadamente criação e gestão de processos, bem como mecanismos básicos de sincronização com threads.

OBS: As respostas das perguntas do laboratório estão no final de cada seção nível 2 que contém perguntas.

2. Experiências Realizadas

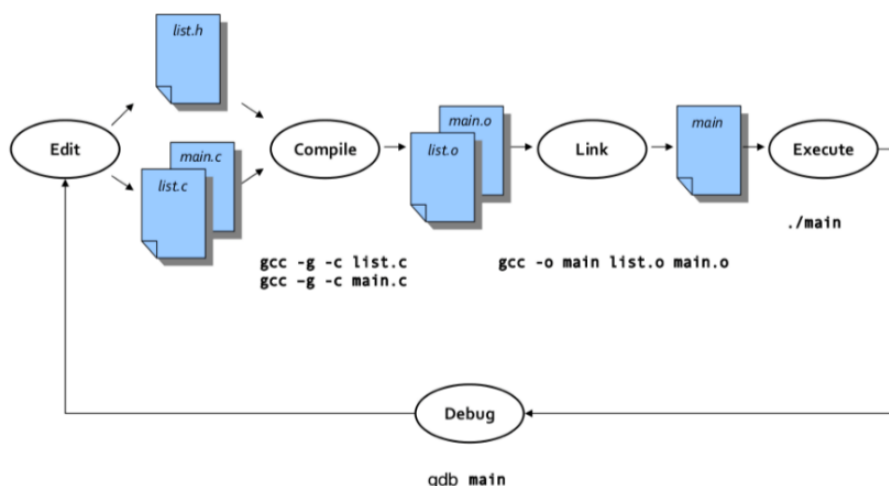


Figura 1 - Ciclo de desenvolvimento de aplicações em linguagem C no ambiente Unix

2.1. Geração do Executável (gcc)

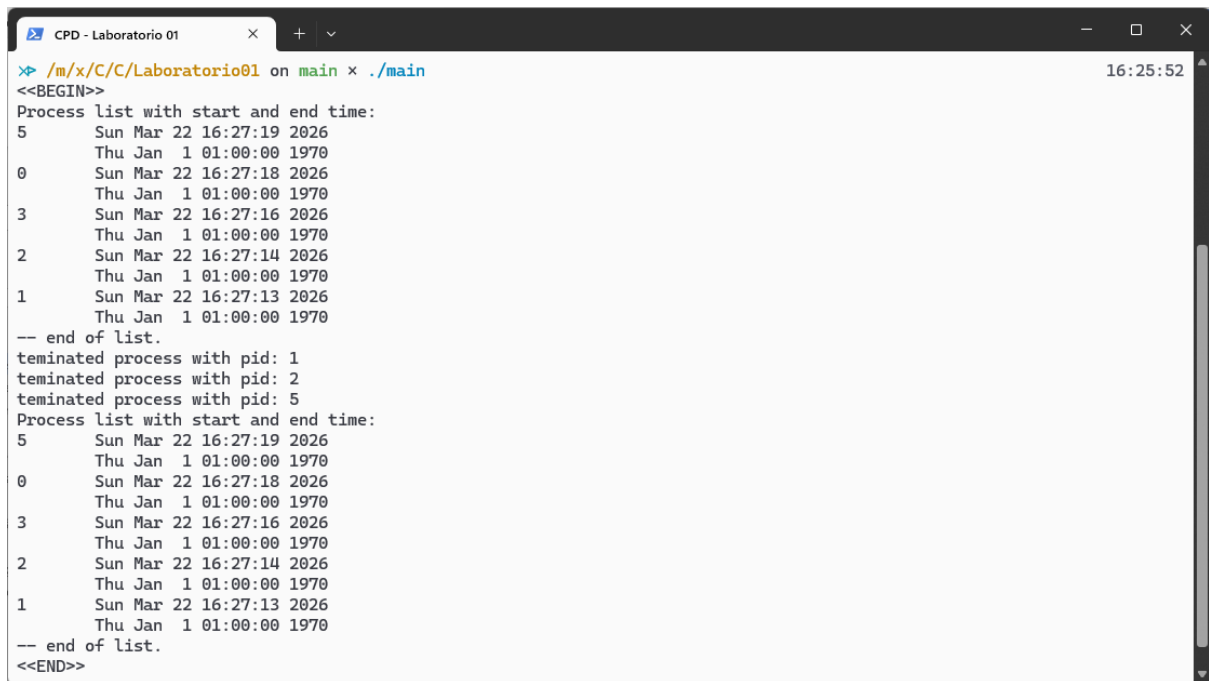
O compilador gcc (GNU Compiler Collection) é uma das ferramentas centrais no desenvolvimento em linguagem C no ambiente Unix. A sua função consiste em traduzir código fonte em linguagem de alto nível para código máquina executável, passando por várias etapas como pré-processamento, compilação e ligação (linking).

Durante o laboratório, utilizou-se o gcc para compilar separadamente os ficheiros fonte, gerando ficheiros objeto (.o), e posteriormente realizar o processo de ligação para gerar o executável final. Este processo permitiu compreender a separação entre compilação e linking.



```
CPD - Laboratorio 01
> /m/x/C/C/Laboratorio01 on main x ls 16:23:45
'Emanuel dos Santos - Memorando.docx'* README.md* aulal-egl.tgz*
> /m/x/C/C/Laboratorio01 on main x tar -zxvf aulal-egl.tgz 16:23:47
./._list.c
list.c
./._list.h
list.h
./._main.c
main.c
> /m/x/C/C/Laboratorio01 on main x gcc -g -c list.c main.c 16:24:06
> /m/x/C/C/Laboratorio01 on main x ls 16:24:21
'Emanuel dos Santos - Memorando.docx'* README.md* aulal-egl.tgz* list.c* list.h* list.o* main.c* main.o*
> /m/x/C/C/Laboratorio01 on main x gcc -o main list.o main.o 16:24:23
> /m/x/C/C/Laboratorio01 on main x | 16:25:52
```

Figura 2 - Processo de Compilação de um programa



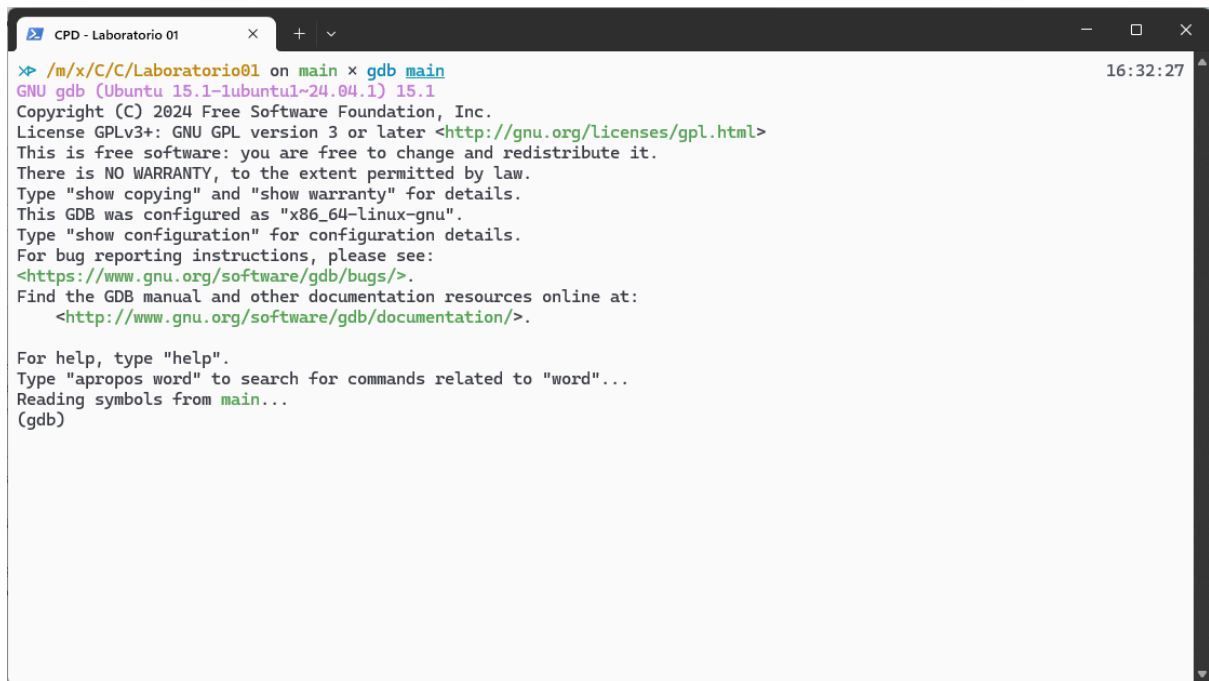
```
>> /m/x/C/C/Laboratorio01 on main x ./main 16:25:52
<<BEGIN>>
Process list with start and end time:
5      Sun Mar 22 16:27:19 2026
      Thu Jan 1 01:00:00 1970
0      Sun Mar 22 16:27:18 2026
      Thu Jan 1 01:00:00 1970
3      Sun Mar 22 16:27:16 2026
      Thu Jan 1 01:00:00 1970
2      Sun Mar 22 16:27:14 2026
      Thu Jan 1 01:00:00 1970
1      Sun Mar 22 16:27:13 2026
      Thu Jan 1 01:00:00 1970
-- end of list.
teminated process with pid: 1
teminated process with pid: 2
teminated process with pid: 5
Process list with start and end time:
5      Sun Mar 22 16:27:19 2026
      Thu Jan 1 01:00:00 1970
0      Sun Mar 22 16:27:18 2026
      Thu Jan 1 01:00:00 1970
3      Sun Mar 22 16:27:16 2026
      Thu Jan 1 01:00:00 1970
2      Sun Mar 22 16:27:14 2026
      Thu Jan 1 01:00:00 1970
1      Sun Mar 22 16:27:13 2026
      Thu Jan 1 01:00:00 1970
-- end of list.
<<END>>
```

Figura 3 - Execução de um programa

2.2. Utilização do Debugger gdb

O gdb (GNU Debugger) é uma ferramenta fundamental para análise e diagnóstico de programas em execução. Permite observar o comportamento interno do programa, inspecionar variáveis e identificar erros como segmentation faults, que são comuns em linguagens de baixo nível como C.

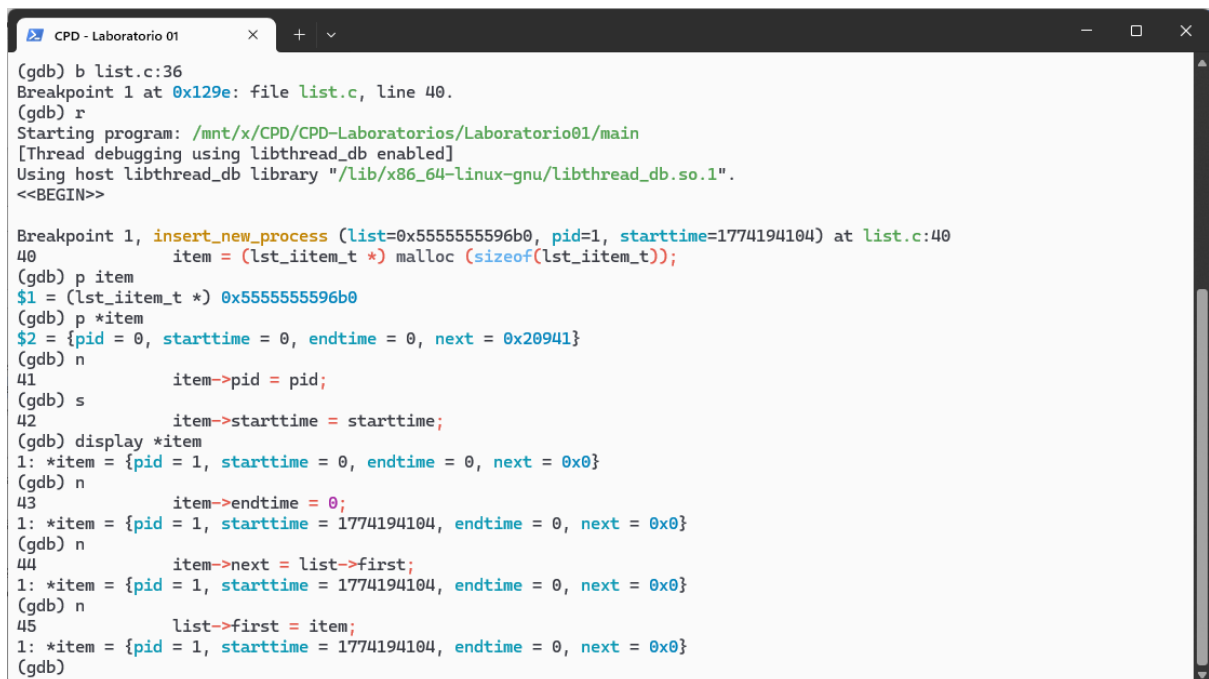
A principal vantagem do gdb reside na possibilidade de executar o programa de forma controlada, permitindo interromper a execução em pontos específicos (breakpoints) e analisar o estado da memória.



```
CPD - Laboratorio 01
>> /m/x/C/Laboratorio01 on main x gdb main
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from main...
(gdb)
```

Figura 4 – Executar um programa em modo de depuração com o gdb



```
CPD - Laboratorio 01
(gdb) b list.c:36
Breakpoint 1 at 0x129e: file list.c, line 40.
(gdb) r
Starting program: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/main
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
<<BEGIN>>

Breakpoint 1, insert_new_process (list=0x5555555596b0, pid=1, starttime=1774194104) at list.c:40
40      item = (lst_iitem_t *) malloc (sizeof(lst_iitem_t));
(gdb) p item
$1 = (lst_iitem_t *) 0x5555555596b0
(gdb) p *item
$2 = {pid = 0, starttime = 0, endtime = 0, next = 0x20941}
(gdb) n
41      item->pid = pid;
(gdb) s
42      item->starttime = starttime;
(gdb) display *item
1: *item = {pid = 1, starttime = 0, endtime = 0, next = 0x0}
(gdb) n
43      item->endtime = 0;
1: *item = {pid = 1, starttime = 1774194104, endtime = 0, next = 0x0}
(gdb) n
44      item->next = list->first;
1: *item = {pid = 1, starttime = 1774194104, endtime = 0, next = 0x0}
(gdb) n
45      list->first = item;
1: *item = {pid = 1, starttime = 1774194104, endtime = 0, next = 0x0}
(gdb)
```

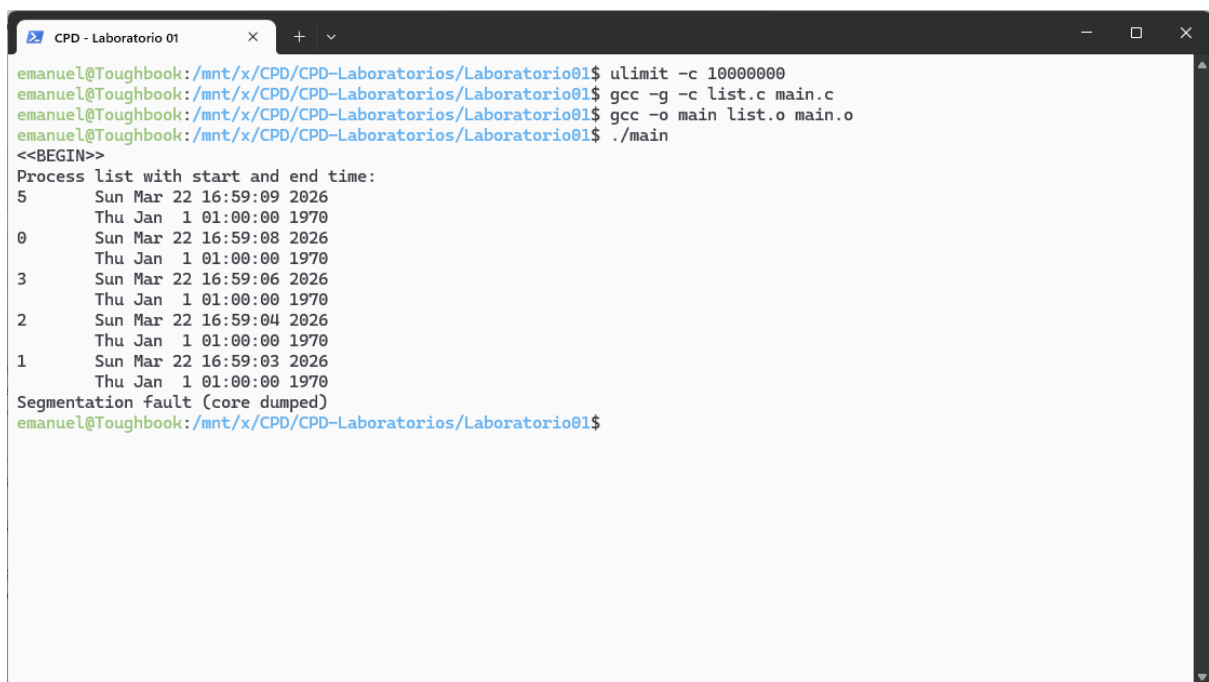
Figura 5 - Depurando um programa com o depurador gdb

```

55 void lst_print(list_t *list)
56 {
57     lst_iitem_t *item;
58
59     printf("Process list with start and end time:\n");
60     item = list->first;
61     while(1){ /* use it only to demonstrate gdb potencial */
62         // while (item != NULL){
63             printf("%d\t%s", item->pid, ctime(&(item->starttime)));
64             printf("\t%s", ctime(&(item->endtime)));
65             item = item->next;
66         }
67     printf("-- end of list.\n");
68 }
69

```

Figura 6 - Descomentando a linha 61 do arquivo list.c



```

CPD - Laboratorio 01
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ ulimit -c 10000000
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ gcc -g -c list.c main.c
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ gcc -o main list.o main.o
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ ./main
<<BEGIN>>
Process list with start and end time:
5      Sun Mar 22 16:59:09 2026
      Thu Jan  1 01:00:00 1970
0      Sun Mar 22 16:59:08 2026
      Thu Jan  1 01:00:00 1970
3      Sun Mar 22 16:59:06 2026
      Thu Jan  1 01:00:00 1970
2      Sun Mar 22 16:59:04 2026
      Thu Jan  1 01:00:00 1970
1      Sun Mar 22 16:59:03 2026
      Thu Jan  1 01:00:00 1970
Segmentation fault (core dumped)
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$

```

Figura 7 - Causando um Segmentation fault (core dumped)

```
CPD - Laboratorio 01
<<BEGIN>>
Process list with start and end time:
5      Sun Mar 22 16:59:09 2026
      Thu Jan 1 01:00:00 1970
0      Sun Mar 22 16:59:08 2026
      Thu Jan 1 01:00:00 1970
3      Sun Mar 22 16:59:06 2026
      Thu Jan 1 01:00:00 1970
2      Sun Mar 22 16:59:04 2026
      Thu Jan 1 01:00:00 1970
1      Sun Mar 22 16:59:03 2026
      Thu Jan 1 01:00:00 1970
Segmentation fault (core dumped)
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ exit
exit
>> /m/x/C/C/Laboratorio01 on main x ./main 16:59:23
<<BEGIN>>
Process list with start and end time:
5      Sun Mar 22 16:59:32 2026
      Thu Jan 1 01:00:00 1970
0      Sun Mar 22 16:59:31 2026
      Thu Jan 1 01:00:00 1970
3      Sun Mar 22 16:59:29 2026
      Thu Jan 1 01:00:00 1970
2      Sun Mar 22 16:59:27 2026
      Thu Jan 1 01:00:00 1970
1      Sun Mar 22 16:59:26 2026
      Thu Jan 1 01:00:00 1970
fish: Job 1, './main' terminated by signal SIGSEGV (Address boundary error)
>> /m/x/C/C/Laboratorio01 on main x 16:59:33
```

Figura 8 - A diferença entre os interpretadores bash e fish ao encerrar um programa pelo sinal SIGSEGV

O Ubuntu estava a redirecionar os core dumps para o systemd.

```
CPD - Laboratorio 01
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ ls
'Emanuel dos Santos - Memorando.docx'  README.md  aulal-egl.tgz  list.c  list.h  list.o  main  main.c  main.o
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ echo core | sudo tee /proc/sys/kernel/core_pattern
[sudo] password for emmanuel:
core
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ ./main
<<BEGIN>>
Process list with start and end time:
5      Sun Mar 22 17:11:10 2026
      Thu Jan 1 01:00:00 1970
0      Sun Mar 22 17:11:09 2026
      Thu Jan 1 01:00:00 1970
3      Sun Mar 22 17:11:07 2026
      Thu Jan 1 01:00:00 1970
2      Sun Mar 22 17:11:05 2026
      Thu Jan 1 01:00:00 1970
1      Sun Mar 22 17:11:04 2026
      Thu Jan 1 01:00:00 1970
Segmentation fault
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ ls
'Emanuel dos Santos - Memorando.docx'  aulal-egl.tgz  list.c  list.o  main.c
README.md  core  list.h  main  main.o
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$
```

Figura 9 - O arquivo core no systemd

```
CPD - Laboratorio 01
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ gdb main core
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from main...
"/mnt/x/CPD/CPD-Laboratorios/Laboratorio01/core" is not a core dump: file format not recognized
(gdb) bt
No stack.
(gdb)
```

Figura 10 - O depurador gdb não reconhece o arquivo core inválido

Mesmo gerando o ficheiro core, o gdb não o reconhece como um core dump file. O problema é que o meu projecto está em /mnt/x/ — uma partição Windows (NTFS). O Linux não consegue escrever um core dump válido em NTFS, por isso o ficheiro fica corrompido.

A solução foi redirecionar o core para uma pasta qualquer na partição Linux:

```
CPD - Laboratorio 01
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ ls
'Emanuel dos Santos - Memorando.docx'  README.md  aulal-egl.tgz  list.c  list.h  main.c
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ ulimit -c 10000000
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ gcc -g -c list.c main.c
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ gcc -o main list.o main.o
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ echo /tmp/core | sudo tee /proc/sys/kernel/core_pattern
[sudo] password for emmanuel:
/tmp/core
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ ./main
<<BEGIN>>
Process list with start and end time:
5      Sun Mar 22 17:31:41 2026
      Thu Jan  1 01:00:00 1970
0      Sun Mar 22 17:31:40 2026
      Thu Jan  1 01:00:00 1970
3      Sun Mar 22 17:31:38 2026
      Thu Jan  1 01:00:00 1970
2      Sun Mar 22 17:31:36 2026
      Thu Jan  1 01:00:00 1970
1      Sun Mar 22 17:31:35 2026
      Thu Jan  1 01:00:00 1970
Segmentation fault (core dumped)
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ ls
'Emanuel dos Santos - Memorando.docx'  README.md  aulal-egl.tgz  list.c  list.h  list.o  main  main.c  main.o
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ ls /tmp
core
snap-private-tmp
systemd-private-5c69c2a2e05d4d4a9acac25076040659-systemd-logind.service-VpvwSc
systemd-private-5c69c2a2e05d4d4a9acac25076040659-systemd-resolved.service-6BDE2S
systemd-private-5c69c2a2e05d4d4a9acac25076040659-systemd-timesyncd.service-mhnepy
```

Figura 11 - Arquivo core válido para o gdb escrito e localizado no linux

```
CPD - Laboratorio 01
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ gdb main /tmp/core
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from main...
[New LWP 1645]
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Core was generated by './main'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  __GI_localtime (t=0x8) at ./time/localtime.c:54

warning: 54      ./time/localtime.c: No such file or directory
(gdb) bt
#0  __GI_localtime (t=0x8) at ./time/localtime.c:54
#1  0x0000748a514de7ed in ctime (t=<optimized out>) at ./time/ctime.c:27
#2  0x00005ac5ad782357 in lst_print (list=0x5ac5e2fff6b0) at list.c:63
#3  0x00005ac5ad7824a6 in main (argc=1, argv=0x7ffc795415b8) at main.c:37
```

Figura 12 - O depurador gdb reconhece o arquivo core e localiza as 3 chamadas da função que gerou o Segmentation fault

```
CPD - Laboratorio 01
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from main...
[New LWP 1645]
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Core was generated by './main'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  __GI_localtime (t=0x8) at ./time/localtime.c:54

warning: 54      ./time/localtime.c: No such file or directory
(gdb) bt
#0  __GI_localtime (t=0x8) at ./time/localtime.c:54
#1  0x0000748a514de7ed in ctime (t=<optimized out>) at ./time/ctime.c:27
#2  0x00005ac5ad782357 in lst_print (list=0x5ac5e2fff6b0) at list.c:63
#3  0x00005ac5ad7824a6 in main (argc=1, argv=0x7ffc795415b8) at main.c:37
(gdb) frame 2
#2  0x00005ac5ad782357 in lst_print (list=0x5ac5e2fff6b0) at list.c:63
63      printf("%d\t%s", item->pid, ctime(&(item->starttime)));
(gdb) p item
$1 = (lst_iitem_t *) 0x0
(gdb) p *item
Cannot access memory at address 0x0
(gdb) |
```

Figura 13 - Usando o frame para encontrar o nível onde ocorreu o erro com o gdb

```
CPD - Laboratorio 01
63      printf("%d\t%s", item->pid, ctime(&(item->starttime)));
1: *item = {pid = 2, starttime = 1774197754, endtime = 0, next = 0x5555555596d0}
(gdb) n
2      Sun Mar 22 17:42:34 2026
64      printf("\t%s", ctime(&(item->endtime)));
1: *item = {pid = 2, starttime = 1774197754, endtime = 0, next = 0x5555555596d0}
(gdb) n
3      Thu Jan 1 01:00:00 1970
65      item = item->next;
1: *item = {pid = 2, starttime = 1774197754, endtime = 0, next = 0x5555555596d0}
(gdb) n
63      printf("%d\t%s", item->pid, ctime(&(item->starttime)));
1: *item = {pid = 1, starttime = 1774197753, endtime = 0, next = 0x0}
(gdb) n

Breakpoint 1, lst_print (list=0x5555555596b0) at list.c:63
63      printf("%d\t%s", item->pid, ctime(&(item->starttime)));
1: *item = {pid = 1, starttime = 1774197753, endtime = 0, next = 0x0}
(gdb) n
4      Sun Mar 22 17:42:33 2026
64      printf("\t%s", ctime(&(item->endtime)));
1: *item = {pid = 1, starttime = 1774197753, endtime = 0, next = 0x0}
(gdb) n
5      Thu Jan 1 01:00:00 1970
65      item = item->next;
1: *item = {pid = 1, starttime = 1774197753, endtime = 0, next = 0x0}
(gdb) n
63      printf("%d\t%s", item->pid, ctime(&(item->starttime)));
1: *item = <error: Cannot access memory at address 0x0>
(gdb)
```

Figura 14 - Variável nula encontrada que provocou o erro de Segmentation fault

2.2.1. Perguntas

1) Qual a diferença entre estes dois comandos?

\$ p item

ou

\$ p *item

R:

O comando p item mostra o valor do próprio ponteiro, ou seja, o endereço de memória (em hexadecimal) para o qual a variável item aponta. Por exemplo: \$1 = (lst_iitem_t *) 0x5555555596b0.

O comando p *item desreferencia o ponteiro e exibe o conteúdo completo da estrutura lst_iitem_t para a qual item aponta, mostrando todos os campos internos: pid, starttime, endtime e next. Por exemplo: \$2 = {pid = 0, starttime = 0, endtime = 0, next = 0x20941}. Este segundo comando é mais útil para inspecionar os dados da estrutura sem precisar de navegar manualmente pelo endereço de memória.

2) O gdb é uma muita boa ferramenta para saber o que aconteceu. Para isso, comece por listar os ficheiros que estão na diretoria, o que observa:

\$ ls

R:

Após o programa terminar com segmentation fault (e tendo configurado ulimit -c 10000000), observa-se um novo ficheiro chamado core. Este ficheiro é gerado automaticamente pelo sistema operativo quando um processo termina abruptamente com um sinal SIGSEGV. Contém uma imagem do estado da memória do processo no momento do erro, incluindo a pilha de chamadas, os registos e os dados das variáveis, e pode ser analisado com o gdb usando: gdb main core.

2.3. Utilização da Ferramenta make

O make é uma ferramenta de automação de compilação que permite gerir dependências entre ficheiros e otimizar o processo de build.

O make baseia-se no conceito de dependências e timestamps, recompilando apenas os ficheiros que sofreram alterações.

Durante o laboratório, verificou-se que:

- apenas ficheiros modificados são recompilados
- alterações em headers afetam múltiplos ficheiros
- regras como clean permitem limpar artefactos de compilação

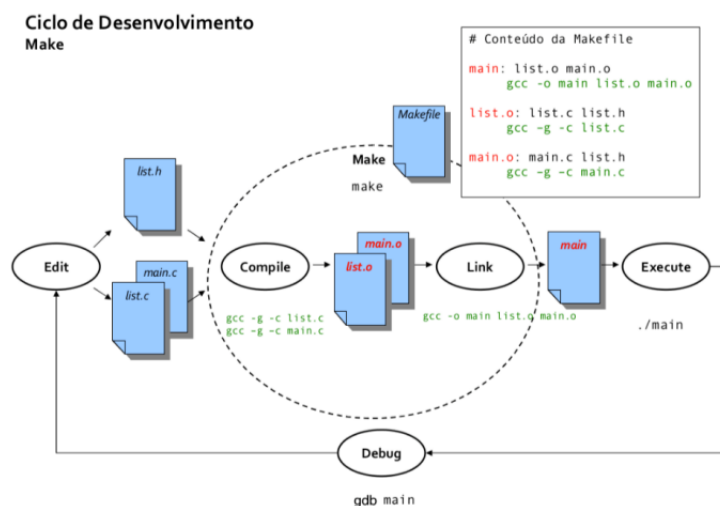


Figura 15 - Ciclo de Compilação com o Make

```
Laboratorio01 >  Makefile
1  main: list.o main.o
2      gcc -o main list.o main.o
3
4  list.o: list.c list.h
5      gcc -g -c list.c
6
7  main.o: main.c list.h
8      gcc -g -c main.c
9
```

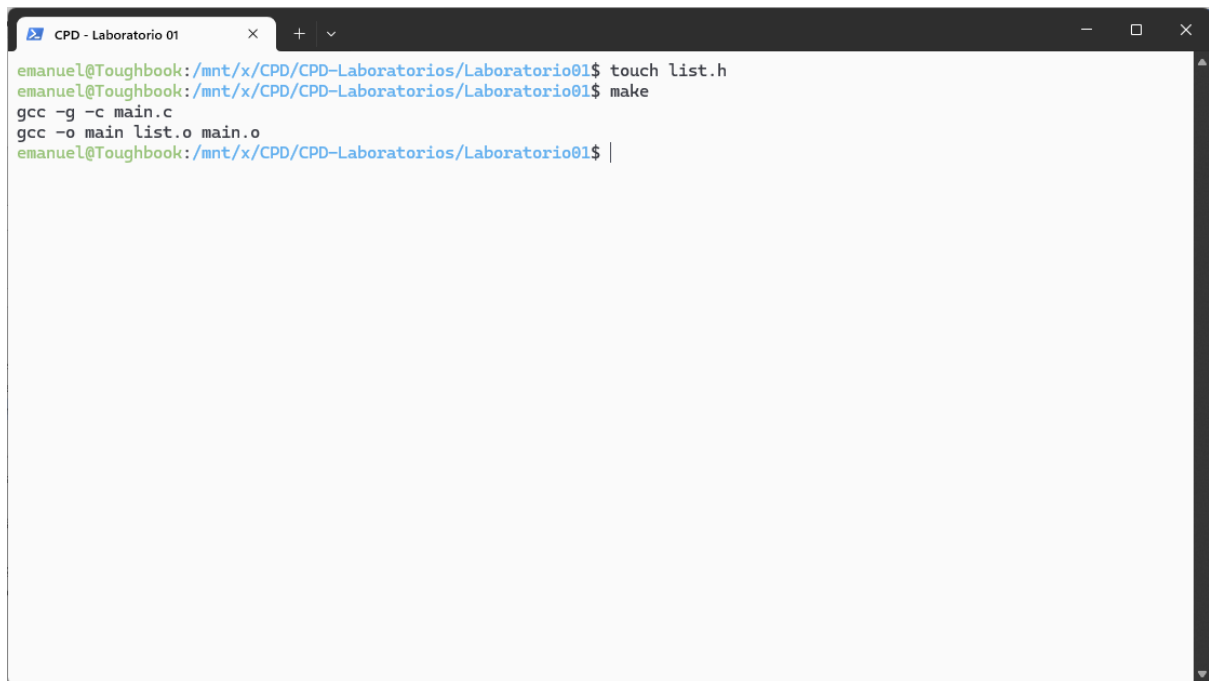
Figura 16 - Ficheiro Makefile configurado para compilar um programa

```
CPD - Laboratorio 01
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ ls
'Emanuel dos Santos - Memorando.docx'  README.md    list.c  list.o  main.c
Makefile                                aulal-egl.tgz list.h  main   main.o
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ make
make: 'main' is up to date.
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ rm list.o
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ make
gcc -g -c list.c
gcc -o main list.o main.o
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ touch main.c
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ make
gcc -g -c main.c
gcc -o main list.o main.o
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ touch list.h
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ make
gcc -g -c list.c
gcc -g -c main.c
gcc -o main list.o main.o
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ touch list.o
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ make
gcc -o main list.o main.o
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ |
```

Figura 17 - Executando comandos make e modificando os arquivos para compilação

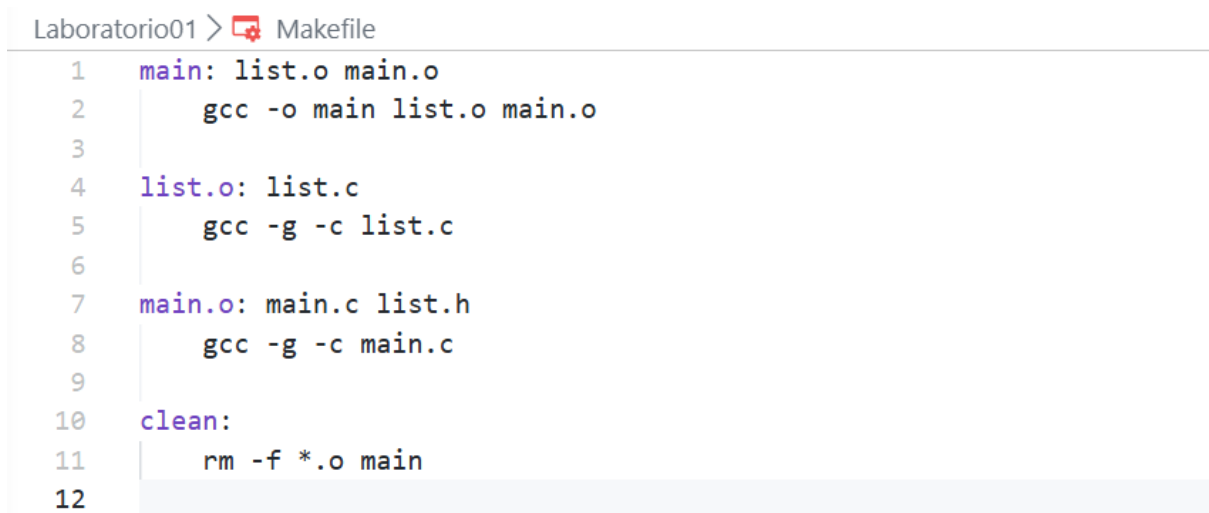
```
Laboratorio01 > Makefile
1  main: list.o main.o
2      gcc -o main list.o main.o
3
4  list.o: list.c
5      gcc -g -c list.c
6
7  main.o: main.c list.h
8      gcc -g -c main.c
9
```

Figura 18 - Retirando a dependência do ficheiro list.h da regra list.o da Makefile.



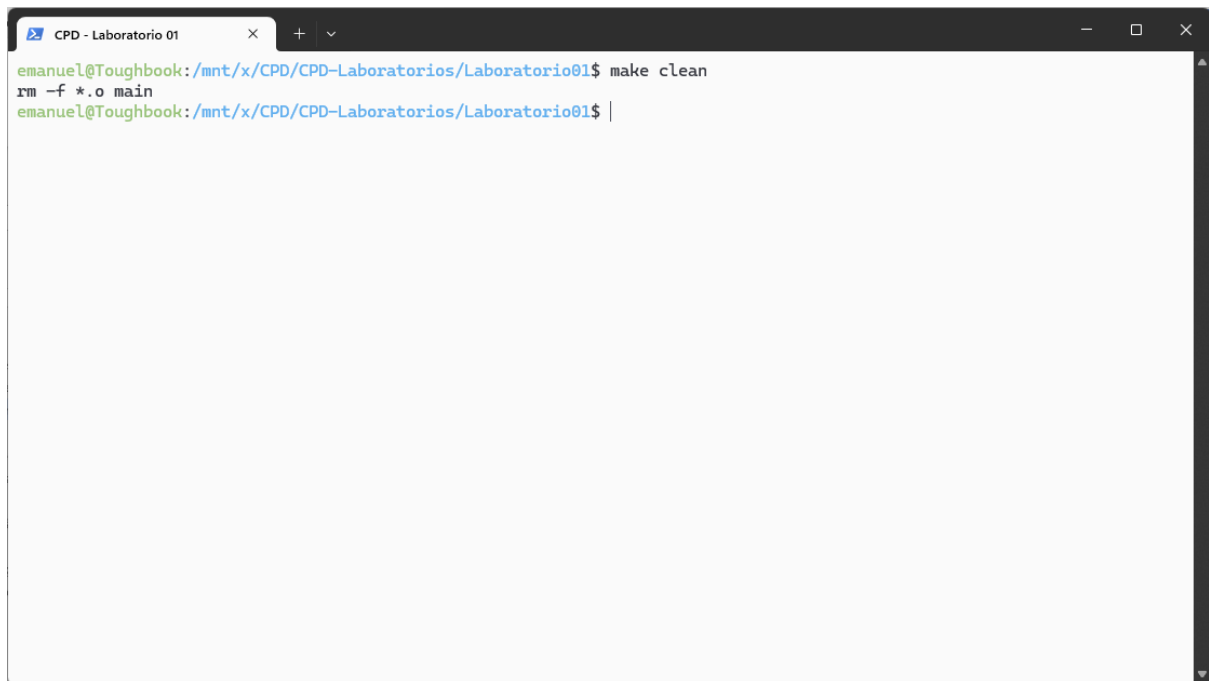
```
CPD - Laboratorio 01
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ touch list.h
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ make
gcc -g -c main.c
gcc -o main list.o main.o
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$
```

Figura 19 - Simulada alteração no ficheiro `list.h` e executando `make`



```
Laboratorio01 > Makefile
1  main: list.o main.o
2      gcc -o main list.o main.o
3
4  list.o: list.c
5      gcc -g -c list.c
6
7  main.o: main.c list.h
8      gcc -g -c main.c
9
10 clean:
11     rm -f *.o main
12
```

Figura 20 - Adicionando a regra `clean` no fim do ficheiro `Makefile`

A terminal window titled "CPD - Laboratorio 01" is shown. The prompt is "emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01\$". The user enters the command "make clean". The terminal output shows "rm -f *.o main" followed by a new prompt "emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01\$".

```
CPD - Laboratorio 01
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$ make clean
rm -f *.o main
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01$
```

Figura 21 - Executando a nova regra com make clean

2.3.1. Perguntas

a) Crie o ficheiro Makefile na sua área de trabalho e execute make. O que aconteceu?

R: Apresentou a mensagem “make: ‘main’ is up to date.”

b) Apague o ficheiro list.o. Re-execute make. Interprete o sucedido.

R:

O make detetou que list.o estava em falta. Como list.o é uma dependência de main, o make recompilou automaticamente list.c para gerar o ficheiro list.o em falta (executando gcc -g -c list.c). Em seguida, realizou novamente o linking para produzir o executável main (gcc -o main list.o main.o). Isto demonstra que o make reconstrói apenas o que é necessário para satisfazer as dependências.

c) Simule uma alteração ao ficheiro main.c com o comando seguinte e reexecute make. Compreenda o resultado.

```
$ touch main.c
```

R:

O comando touch main.c atualizou o timestamp de main.c, tornando-o artificialmente mais recente do que main.o. O make detetou que main.o estava desatualizado relativamente a main.c e recompilou apenas main.c (gcc -g -c main.c), gerando um novo main.o. Em seguida, refez o linking para produzir o executável main atualizado. Note-se que list.c não foi recompilado, pois a sua dependência não foi alterada.

d) Simule a alteração do ficheiro list.h e execute make. Porque razão todos os ficheiros foram gerados?

R:

O ficheiro list.h é declarado como dependência tanto de list.o como de main.o no Makefile (ambas as regras incluem list.h). Quando o timestamp de list.h é atualizado com touch list.h, ambos os ficheiros objeto ficam desatualizados relativamente ao header. O make força a recompilação de list.c e de main.c, e em seguida refaz o

linking. Isto é o comportamento correto: se um header partilhado mudar, todos os ficheiros que o incluem devem ser recompilados para garantir consistência.

- e) Simule a alteração do ficheiro list.o. O que acontece quando faz make list.o? E se agora fizer make?

R:

make list.o: O touch list.o atualizou o timestamp de list.o, tornando-o mais recente do que os seus ficheiros fonte (list.c e list.h). O make verificou que list.o era mais recente do que as suas dependências e não fez nada, apresentando “make: ‘list.o’ is up to date.”

make (sem alvo): O make verificou que main era mais antigo do que list.o (que acabou de ser tocado). Como list.o é mais recente do que main, o make realizou apenas o passo de linking (gcc -o main list.o main.o), sem recompilar nenhum ficheiro .c. Isto mostra que o make opera ao nível dos timestamps, independentemente de o conteúdo ter mudado ou não.

- f) Retire a dependência do ficheiro list.h da regra list.o da Makefile. Repita procedimento da alínea d). Explique a diferença no resultado?

R:

Sem list.h como dependência na regra list.o, o make já não sabe que list.o depende desse header. Ao repetir o procedimento da alínea d) (touch list.h + make), apenas main.o é recompilado (pois main.o ainda declara list.h como dependência), mas list.o não é recompilado. A diferença é que só são gerados main.o e main, ao invés de todos os ficheiros como na alínea d). Isto representa um erro de gestão de dependências: se list.h for alterado com mudanças que afetam list.c, o list.o ficará inconsistente e o binário poderá ter comportamento indefinido.

- g) Adicione a regra seguinte no fim do ficheiro. O que descreve esta regra? Identifique: o alvo, as dependências e o comando. Tenha em atenção que os espaços iniciais em cada linha são tabs.

```
clean:
    rm -f *.o main
```

R:

- h) Execute make clean. O que aconteceu? Porque razão o comando é executado sempre que esta regra é invocada explicitamente?

R:

Esta regra descreve uma ação de limpeza do projeto:

- Alvo: clean
- Dependências: nenhuma (a regra não depende de nenhum ficheiro)
- Comando: rm -f *.o main (remove todos os ficheiros objeto e o executável main)

A regra clean é tipicamente usada para repor o projeto ao estado inicial antes de qualquer compilação, permitindo um build limpo do zero. A flag -f no rm faz com que o comando não apresente erro caso os ficheiros não existam.

2.4. Exercícios

2.4.1. update_terminated_process

A função `update_terminated_process` recebe uma lista, um valor de `pid` e um tempo de fim, procura pelo elemento com esse valor de `pid` e atualiza esse elemento com o tempo de fim.

```
44 void update_terminated_process(list_t *list, int pid, time_t endtime)
45 {
46     lst_iitem_t *item;
47
48     item = list->first;
49     while (item != NULL)
50     {
51         if (item->pid == pid)
52         {
53             item->endtime = endtime;
54             printf("terminated process with pid: %d\n", pid);
55             return;
56         }
57         item = item->next;
58     }
59     printf("Process with pid: %d not found\n", pid);
60 }
```

Figura 22 - Implementada a função `update_terminated_process` no ficheiro `list.c`.

2.4.2. Gestor de Tarefas Pessoais

Implementou-se a aplicação de gestão de tarefas pessoais com recurso a um array de listas ligadas (uma por nível de prioridade, de 0 a 5). Os comandos `new`, `list` e `complete` foram implementados conforme especificado:

- `new <prioridade> <id>`: insere a nova tarefa na lista do nível de prioridade correspondente, com timestamp de criação.
- `list <prioridade>`: itera as listas da prioridade máxima (5) até à prioridade indicada, apresentando primeiro as tarefas mais prioritárias e, entre iguais, as mais recentes.
- `complete <id>`: procura e remove a tarefa com o identificador dado em todas as listas; caso não exista, apresenta "TAREFA INEXISTENTE".

```
Gestor de Tarefas Pessoais x + v
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/ex02$ make
gcc -g -c task.c
gcc -g -c main.c
gcc -o main task.o main.o
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/ex02$ ./main
new 5 estudar_cpd
new 0 ver_msg_whatsapp
new 0 ver_msg_instagram
new 4 jogar_futebol
new 4 jogar_videogame
new 2 dormir
new 2 comer
complete andar
TAREFA INEXISTENTE
list 2
5 estudar_cpd
4 jogar_videogame
4 jogar_futebol
2 comer
2 dormir
complete jogar_futebol
list 2
5 estudar_cpd
4 jogar_videogame
2 comer
2 dormir
quit
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/ex02$
```

Figura 23 - Compilando o programa Gestor de Tarefas Pessoais e adicionando, listando e completando tarefas no programa

2.5. Desafio do Laboratório

```
emanuel@Toughbook: /mnt/ x + v
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/desafio$ make
gcc -g -I ./includes -c main.c
gcc -g -I ./includes -c ./srcs/list.c
gcc -g -I ./includes -c ./srcs/commandlinereader.c
gcc -o cpd-terminal main.o list.o commandlinereader.o -lpthread
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/desafio$ make fibonacci
gcc -g -I ./includes -o fibonacci ./srcs/fibonacci.c
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/desafio$ cat tests/input.txt
fibonacci 10000
fibonacci 10001
fibonacci 10002
exit
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/desafio$ ./cpd-terminal < tests/input.txt
Process 2138 started to generate Fibonacci up to 10002.
Process 2137 started to generate Fibonacci up to 10001.
Process 2136 started to generate Fibonacci up to 10000.
Process 2136 finished Fibonacci series at value: 2788724563990792802.
terminated process with pid: 2136
Process 2137 finished Fibonacci series at value: 15574651946073070043.
terminated process with pid: 2137
Process 2138 finished Fibonacci series at value: 18363376510063862845.
terminated process with pid: 2138
Process list with start and end time:
2138 Sun Mar 22 23:14:13 2026
Sun Mar 22 23:14:16 2026
2137 Sun Mar 22 23:14:13 2026
Sun Mar 22 23:14:16 2026
2136 Sun Mar 22 23:14:13 2026
Sun Mar 22 23:14:16 2026
-- end of list.
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/desafio$
```

Figura 24 - Compilando o programa desafio, o teste fibonacci do input.txt e executando o programa passando pelo stdin os comandos a serem executados paralelamente

```
emanuel@Toughbook: /mnt/ x + v
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/desafio$ cat < tests/input1.txt
fibonacci 5000
exit
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/desafio$ ./cpd-terminal < tests/input1.txt
Process 2180 started to generate Fibonacci up to 5000.
Process 2180 finished Fibonacci series at value: 9720516145911357709.
terminated process with pid: 2180
Process list with start and end time:
2180   Sun Mar 22 23:15:32 2026
      Sun Mar 22 23:15:34 2026
-- end of list.
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/desafio$ cat < tests/input2.txt
nao_existe
fibonacci 3000
exit
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/desafio$ ./cpd-terminal < tests/input2.txt
Error executing: nao_existe
Process 2191 started to generate Fibonacci up to 3000.
Process 2191 finished Fibonacci series at value: 13838119898447193649.
terminated process with pid: 2190
terminated process with pid: 2191
Process list with start and end time:
2191   Sun Mar 22 23:15:50 2026
      Sun Mar 22 23:15:51 2026
2190   Sun Mar 22 23:15:50 2026
      Sun Mar 22 23:15:51 2026
-- end of list.
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/desafio$
```

Figura 25 - Executando novamente o programa com os testes input1.txt e input2.txt

```
emanuel@Toughbook: /mnt/ x + v
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/desafio$ cat tests/input3.txt
exit
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/desafio$ ./cpd-terminal < tests/input3.txt
Process list with start and end time:
-- end of list.
emanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio01/desafio$ |
```

Figura 26 - Executando o programa com o último teste input3.txt

2.5.1.Arquitetura da Solução

A arquitetura do sistema foi concebida com base num modelo de concorrência híbrido, combinando processos e threads:

- **Thread principal:** responsável pela leitura de comandos e criação de processos
- **Thread monitora:** responsável pela detecção da terminação dos processos filhos e recolha de métricas de execução

A comunicação entre estas duas componentes é realizada através de:

- uma lista ligada de processos
- uma variável partilhada (numChildren)
- mecanismos de exclusão mútua (mutex)

2.5.2.Etapa 1 — Comandos do Teclado (fork/exec)

execução de programas foi implementada recorrendo ao modelo clássico *fork-exec* do Unix. Para cada comando introduzido pelo utilizador:

1. É criado um novo processo através da chamada `fork()`
2. O processo filho substitui o seu espaço de execução com `execvp()`
3. O processo pai continua a execução sem bloquear (execução em background)

Este modelo permite a execução concorrente de múltiplos processos, característica essencial para sistemas paralelos.

2.5.3. Etapa 2 — Monitorização com Threads

Para monitorizar a execução dos processos filhos, foi criada uma thread adicional responsável por acompanhar a sua terminação.

Esta thread executa um ciclo contínuo, no qual:

- verifica se existem processos ativos (`numChildren > 0`)
- caso existam, invoca `wait()` para bloquear até à terminação de um processo
- regista o tempo de término utilizando a função `time()`

Caso não existam processos ativos, a thread entra num estado de espera temporizada (`sleep(1)`), evitando consumo excessivo de CPU.

2.5.4. Sincronização e Gestão de Recursos

A informação relativa aos processos é armazenada numa lista simplesmente ligada, contendo:

- PID do processo
- tempo de início
- tempo de término

Esta estrutura permite:

- inserção eficiente de novos processos
- atualização dinâmica aquando da sua terminação
- apresentação final dos resultados

Devido à existência de acesso concorrente a dados partilhados, foi necessário implementar mecanismos de sincronização.

Foi utilizado um *mutex* (`pthread_mutex_t`) para garantir exclusão mútua no acesso às seguintes variáveis:

- lista de processos
- contador `numChildren`

Este mecanismo previne condições de corrida (*race conditions*) e assegura a consistência dos dados.

Quando o utilizador introduz o comando `exit`, o sistema:

1. sinaliza a thread monitora através da variável `done`
2. aguarda a terminação de todos os processos filhos
3. sincroniza com a thread monitora (`pthread_join`)
4. apresenta o relatório final com PID e tempo de execução

Este processo garante uma terminação ordeira e consistente do sistema.

3. Desafios

Foi observado que a execução de comandos utilizando `execvp` depende da localização do executável no sistema de ficheiros e da variável de ambiente `PATH`.

Inicialmente, a execução do programa `fibonacci` falhava, uma vez que o executável não se encontrava no diretório corrente nem em nenhum diretório presente no `PATH`.

Após a compilação do programa no diretório do projeto, a execução passou a funcionar corretamente.

Para tornar o sistema mais robusto, foi implementado um mecanismo de fallback que tenta executar o comando no diretório atual (./) caso a resolução via PATH não seja bem-sucedida.

4. Referências Bibliográficas

Computação Paralela e Distribuída - Introdução ao Ambiente Unix – 2025/2026 por João Costa – Laboratório 01

CodeQuoi - Threads, Mutexes and Concurrent Programming in C por Mia Combeau.

CodeQuoi - Creating and Killing Child Processes in C por Mia Combeau

The Linux Programming Interface (TLPI) por Michael Kerrisk. Chapters 29 to 33 (POSIX Threads and Thread Synchronization), Chapters 24 to 26 (Process Creation, Termination, and Monitoring).

5. Repositório GitHub

Repositório: github.com/SW-Wanted/CPD-Laboratorios.¹

¹ Foi enviado um convite de colaborador ao utilizador [joaojdacosta](#).